FetalAI: Using Machine Learning To Predict And Monitor Fetal Health

Prepared By: Tejas Tanaji Salokhe

Project Guide: Revanth

Organization: SmartBridge

1. Introduction

1.1 Background

Fetal mortality and maternal health complications remain a pressing global healthcare challenge. During the third trimester of pregnancy and labor, continuous monitoring of fetal well-being is critical. Cardiotocography (CTG) is a globally accepted and cost-effective technique used to monitor fetal heart rate (FHR) and uterine contractions (UC). While CTG provides vital physiological data, the manual interpretation of these complex electrocardiograms by obstetricians is highly subjective, leading to inter-observer variability and potential diagnostic errors.

1.2 Problem Statement

Misinterpretation of CTG data can lead to two critical issues: false positives, resulting in unnecessary surgical interventions (like emergency C-sections), or false negatives, which can lead to severe fetal distress or mortality. Therefore, there is a critical need for an automated, objective, and highly accurate decision-support

system to classify fetal health states.

1.3 Objectives

- To develop an automated machine learning model capable of classifying fetal health into three distinct categories: Normal, Suspect, and Pathological.
- To perform rigorous feature engineering and handle severe data imbalance to prevent model bias.
- To deploy the trained predictive model into a real-time, user-friendly web application for healthcare professionals.

2. System Architecture and Dataset

2.1 Dataset Description

The project is built upon the fetal_health.csv dataset, which contains 2,126 records of features extracted from Cardiotocogram exams by expert obstetricians.

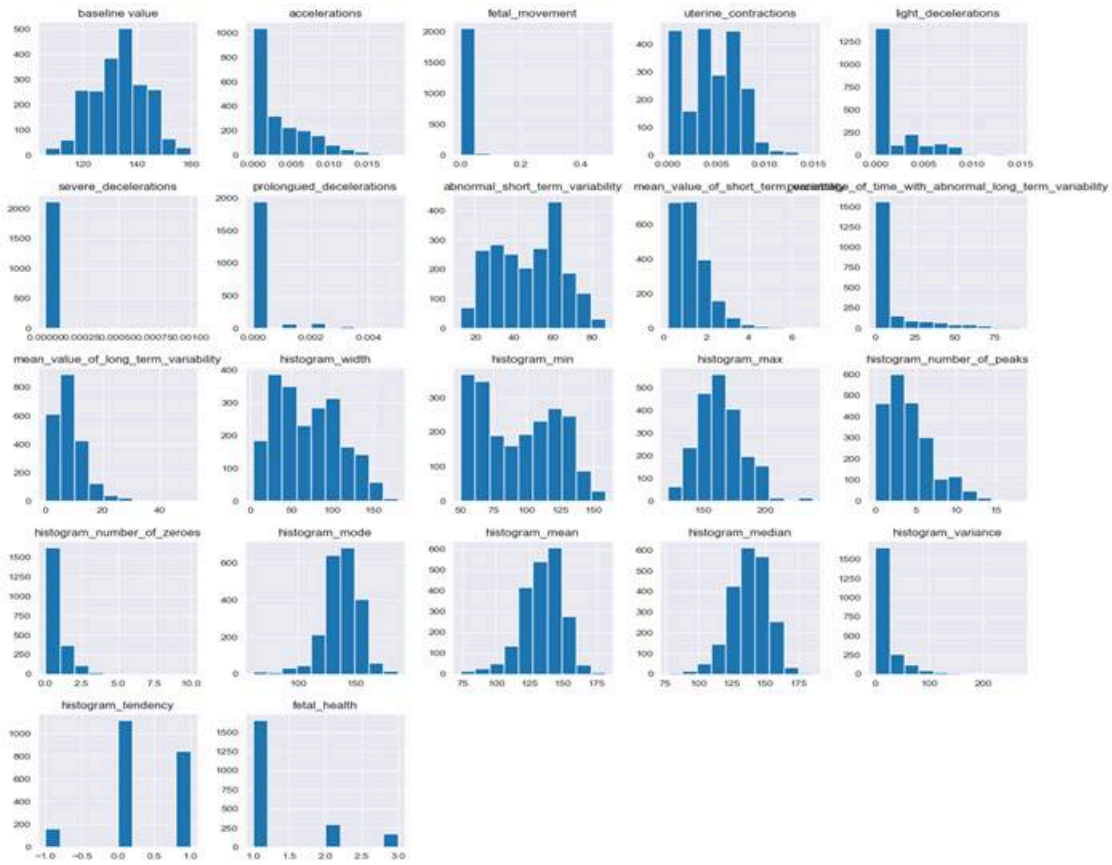
	baseline value	accelerations	fetal_movement	uterine_contractions	light_decelerations	severe_decelerations	prolongued_decelerations	abnormal_short_term_va
0	120.0	0.000	0.0	0.000	0.000	0.0	0.0	
1	132.0	0.006	0.0	0.006	0.003	0.0	0.0	
2	133.0	0.003	0.0	0.008	0.003	0.0	0.0	
3	134.0	0.003	0.0	0.008	0.003	0.0	0.0	
4	132.0	0.007	0.0	0.008	0.000	0.0	0.0	

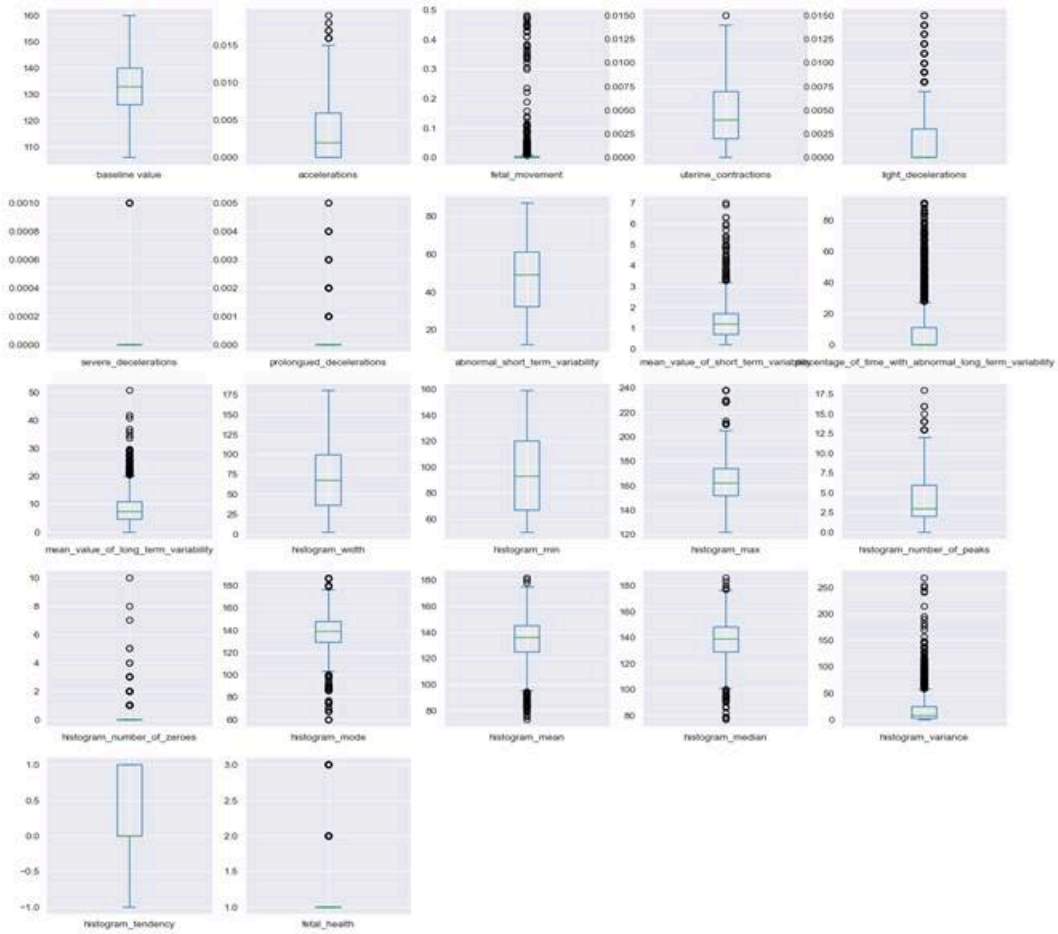
- Size: 2,126 instances and 22 columns.
- Target Variable: fetal_health, categorized as 1.0 (Normal), 2.0 (Suspect), and 3.0 (Pathological).
- Initial Class Distribution: Highly imbalanced.
    - Normal: 1,158 (54.4%)
    - Suspect: 201 (9.4%)
    - Pathological: 129 (6.0%)

2.2 Exploratory Data Analysis (EDA)

Extensive visual and statistical analyses were performed to understand the underlying distributions:

- Univariate Analysis: Histograms and boxplots revealed the presence of outliers in parameters like histogram_variance and accelerations.
- Bivariate Analysis (Correlation Matrix): A Seaborn correlation heatmap was generated to identify feature redundancies and relationships with the





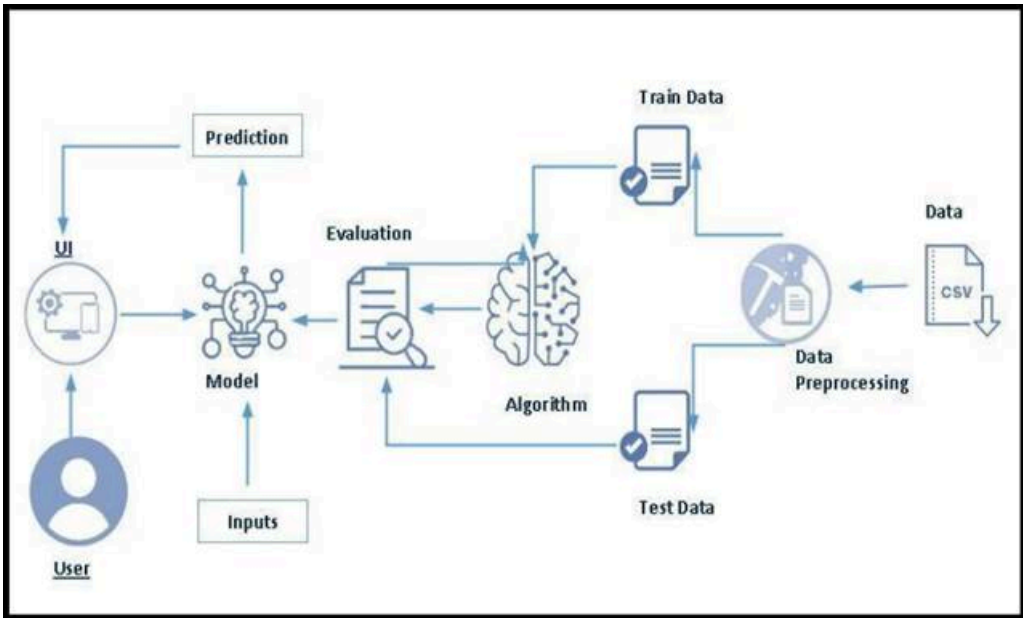
target variable. It was observed that prolongued_decelerations and abnormal_short_term_variability had the highest positive correlation with pathological outcomes, while accelerations had a negative correlation (indicating a healthy fetus).

3. Methodology & Data Preprocessing

3.1 Feature Selection

To optimize computational efficiency and reduce noise (curse of dimensionality), features with negligible impact or high multicollinearity were removed. The model was trained exclusively on the 8 most critical predictive features:

1. prolongued_decelerations
2. abnormal_short_term_variability
3. percentage_of_time_with_abnormal_long_term_variability
4. histogram_variance
5. histogram_median
6. mean_value_of_long_term_variability
7. histogram_mode
8. accelerations



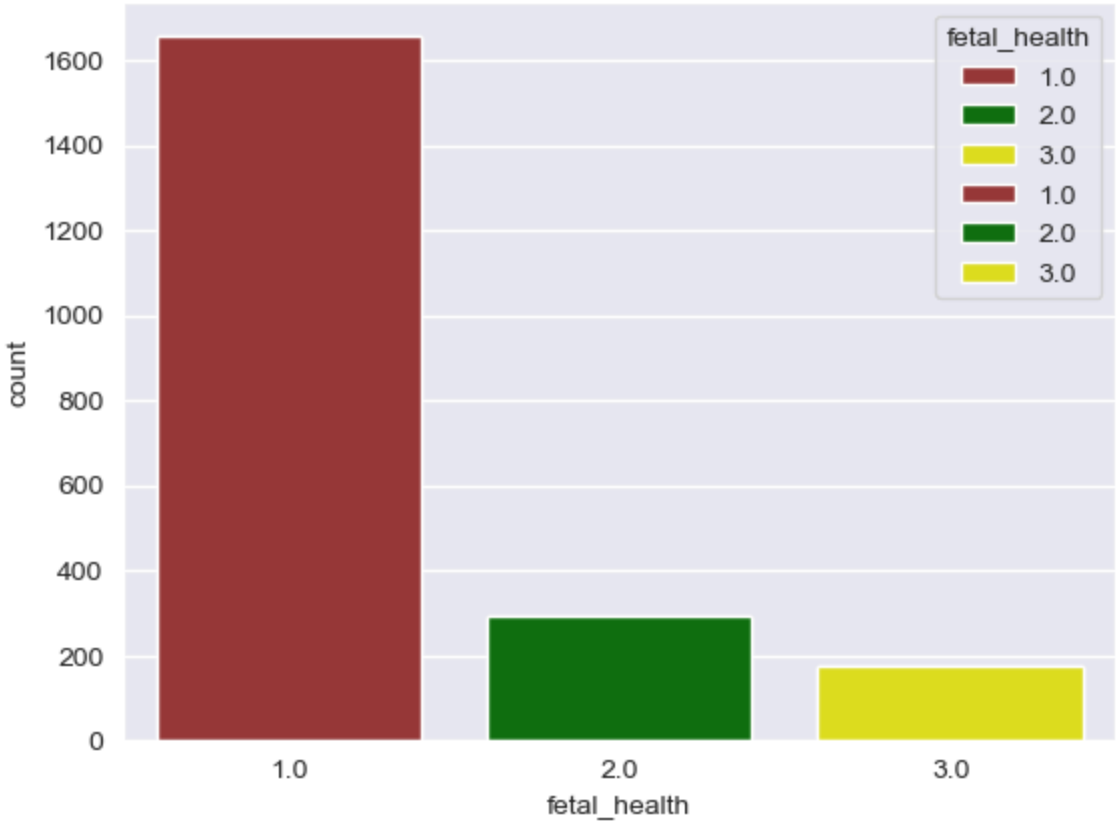
3.2 Feature Scaling

Machine learning algorithms, particularly those relying on distance calculations or gradient descent, perform optimally when features are on a similar scale. A

MinMaxScaler was applied to normalize the selected 8 features, transforming all numerical values into a standardized range between 0 and 1 without distorting the shape of the original distribution.

3.3 Handling Imbalanced Data: SMOTE

Training a model on the initial imbalanced dataset would result in an algorithm that heavily favors the "Normal" majority class, leading to poor identification of "Pathological" cases—which is unacceptable in a medical context. To resolve this, SMOTE (Synthetic Minority Over-sampling Technique) was utilized. SMOTE synthesizes new, realistic examples for the minority classes by interpolating between existing instances in the feature space.

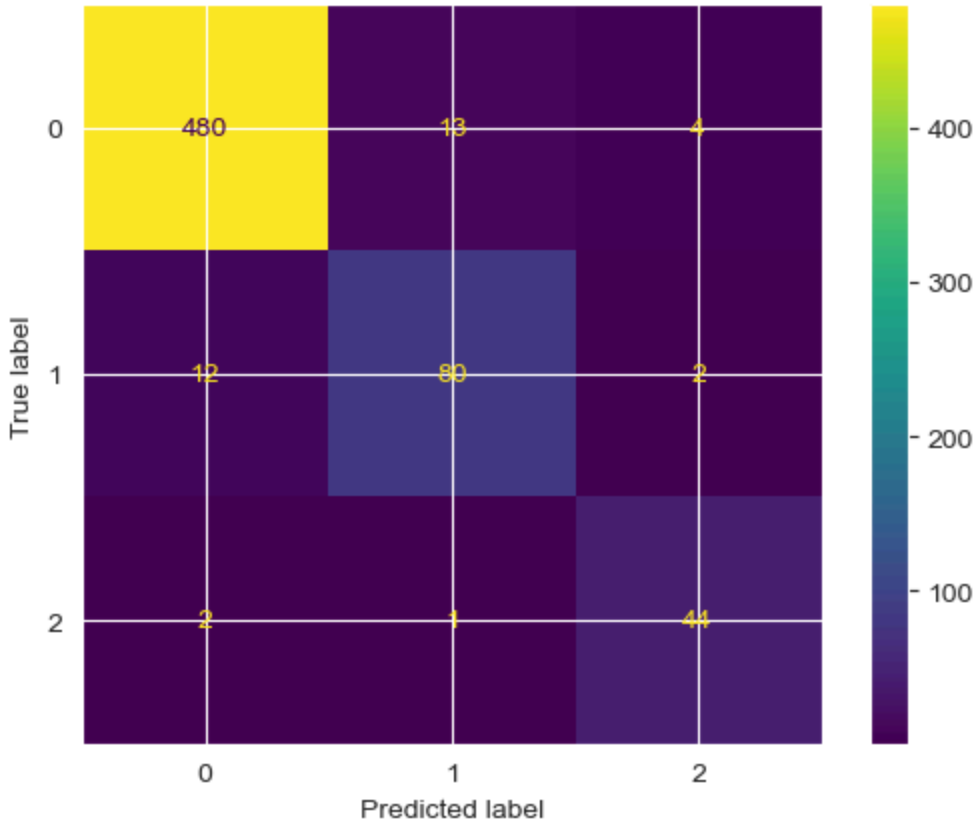


- Data Split: The data was first split into training (70%) and testing (30%) sets.
- Application: SMOTE was applied *only* to the training data to prevent data leakage.
- Result: The training set achieved perfect parity, containing exactly 1,158 instances for all three classes (Total training samples: 3,474).

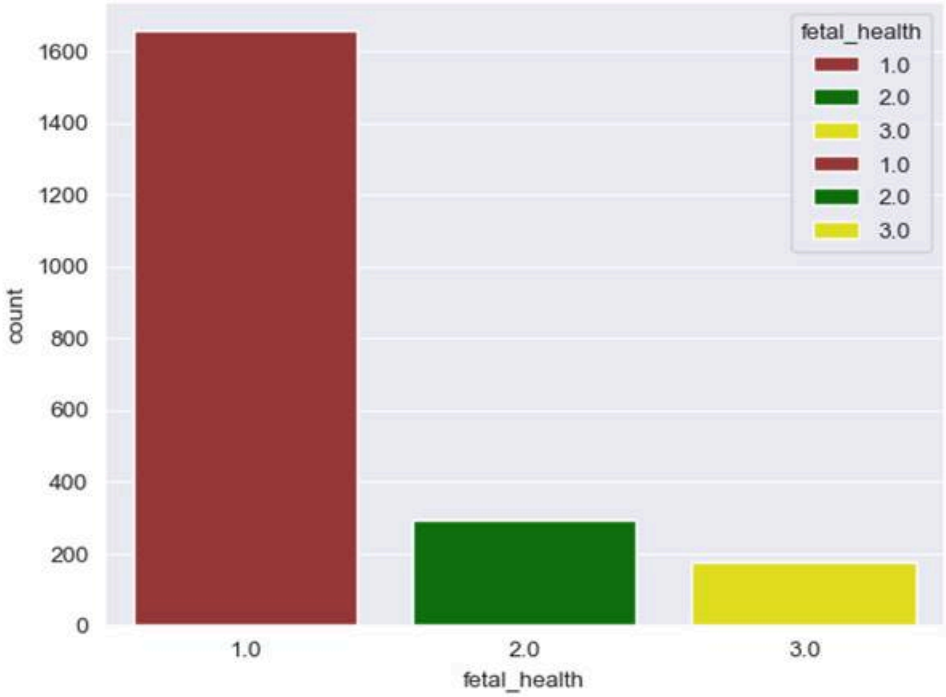
4. Model Development: Random Forest

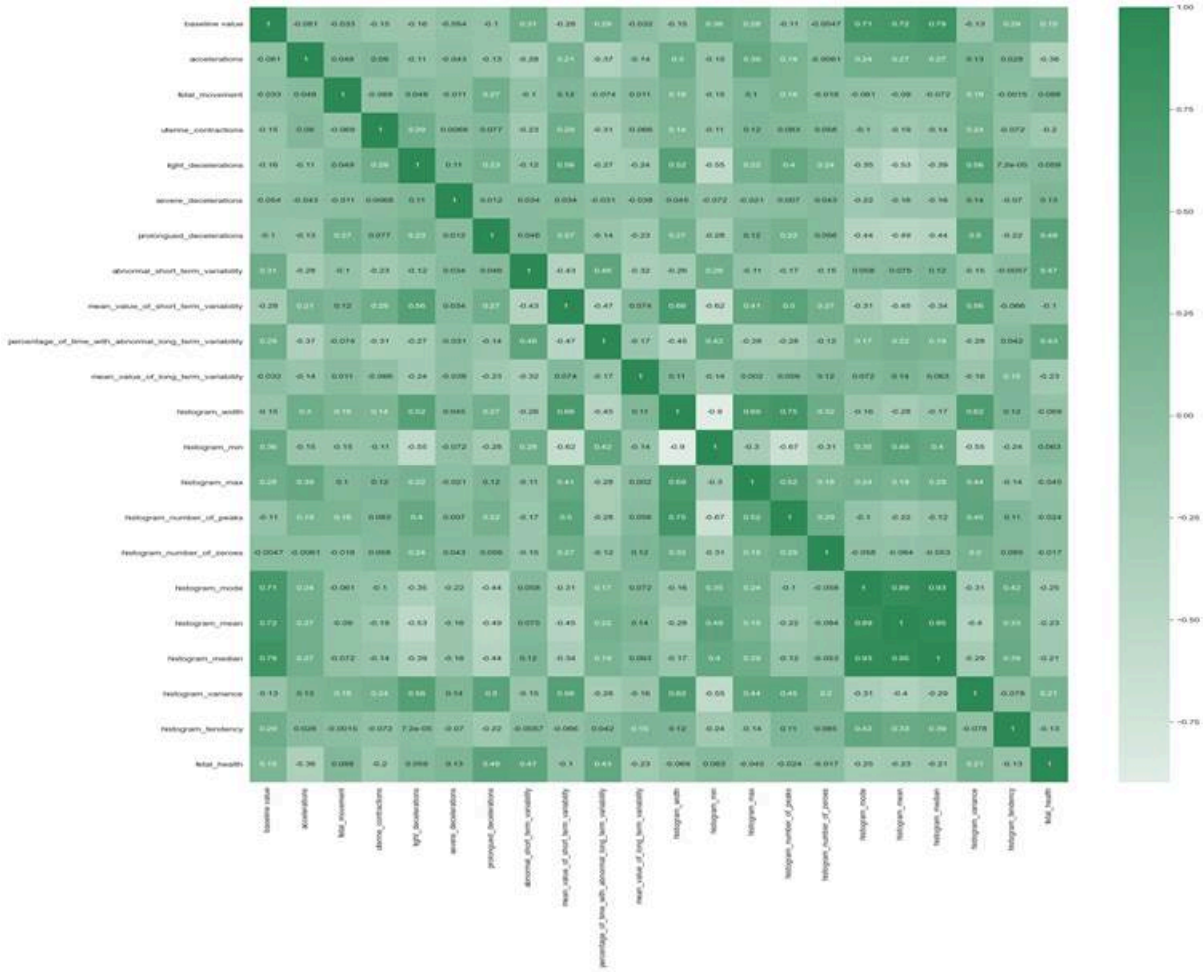
4.1 Algorithm Selection

A Random Forest Classifier was selected as the primary predictive engine. Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the mode of the classes for classification.



- Why Random Forest? Clinical data often contains non-linear relationships. Random Forest handles non-linearity exceptionally well, is highly robust against overfitting (due to bootstrap aggregating or "bagging"), and provides excellent feature importance metrics.





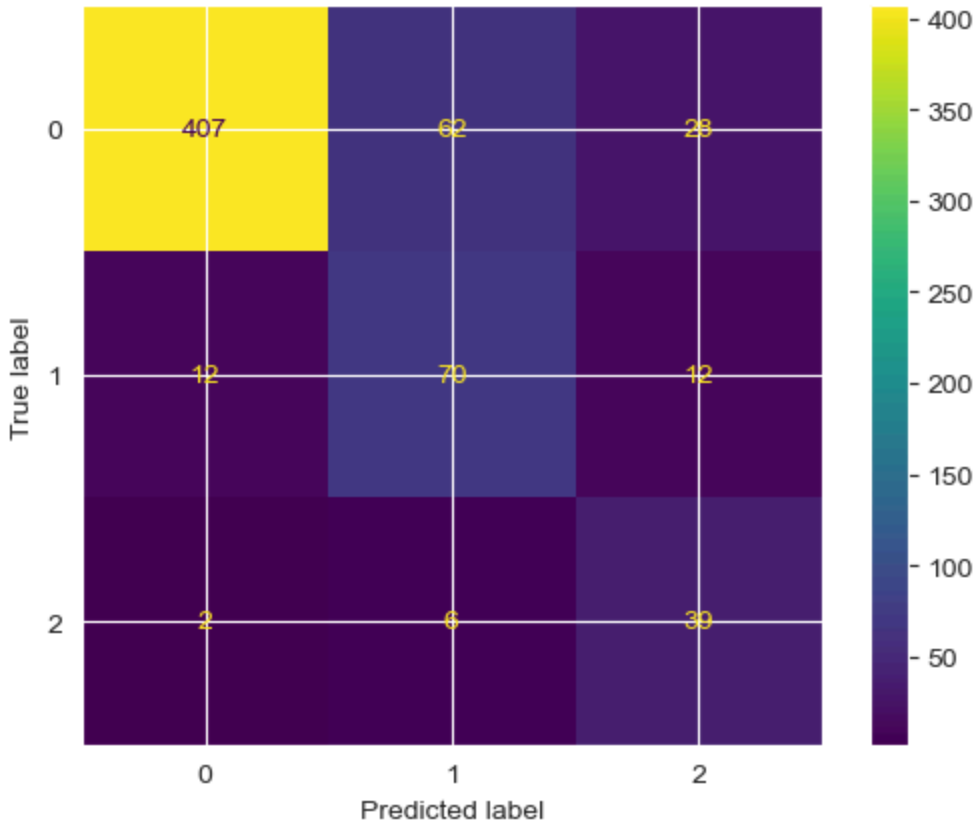
4.2 Model Evaluation and Results

The model was evaluated against the unseen 30% testing dataset.

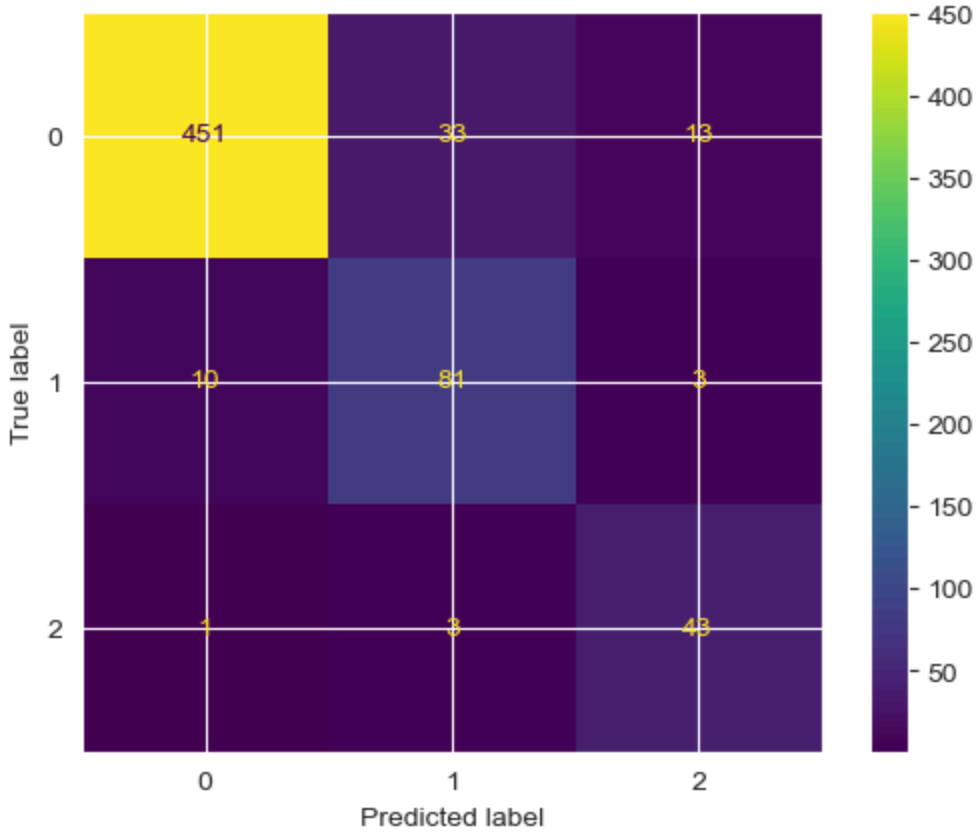
- Overall Accuracy: The model achieved a high accuracy of 94.67%.
- Confusion Matrix: The confusion matrix visualized the model's precision and recall. It demonstrated a highly acceptable True Positive Rate for the "Pathological" and "Suspect" classes. Crucially, the False Negative rate (predicting a sick fetus as healthy) was minimized, which is the most vital metric for patient safety.
- Serialization: The optimal model architecture and weights were serialized into a binary file (fetalAI_model.pkl) using the pickle library, allowing it to

be transferred to the web server without requiring retraining.

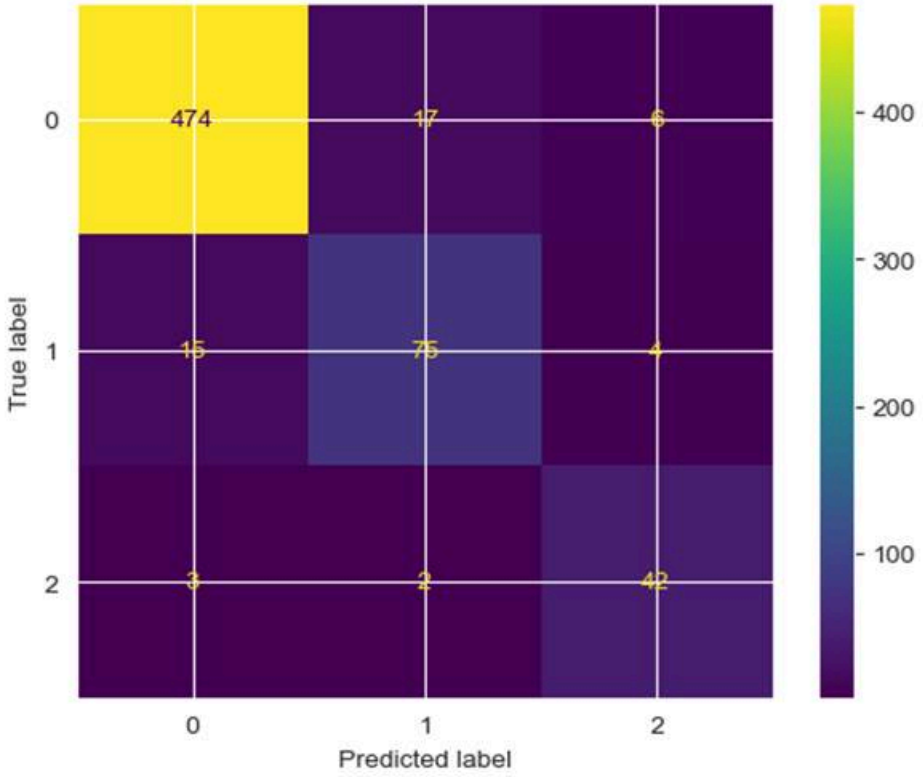
Logistic Regression:



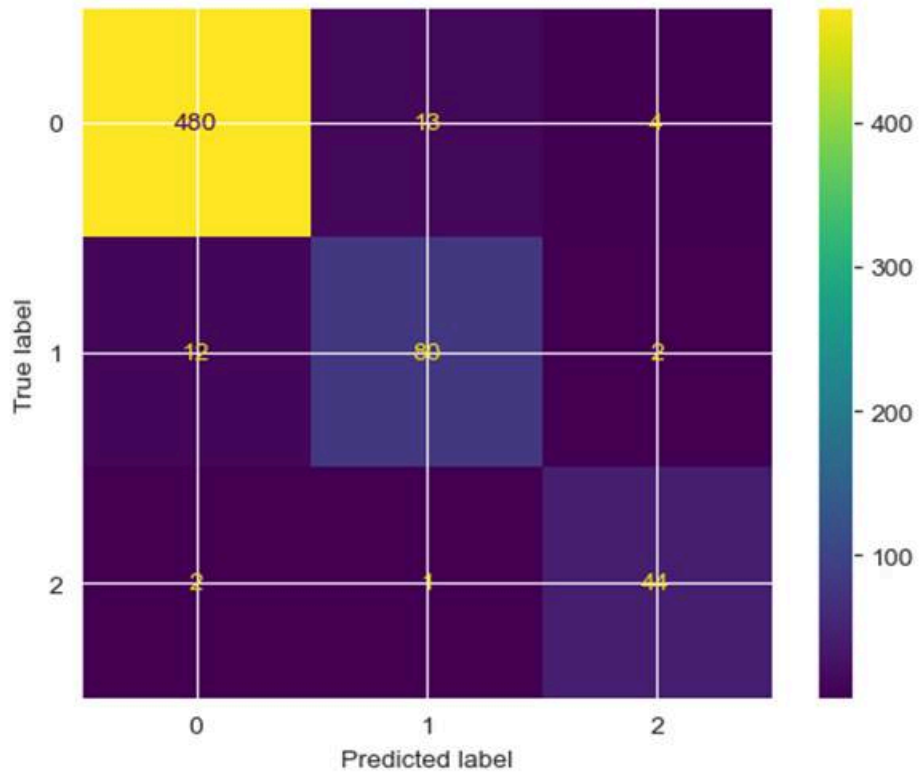
KNeighbors Classifie:



Decisoin Tree Classifier:



Random Forest Classifier



5. Web Application Integration (Flask)

To transition the model from a Jupyter Notebook experiment into a usable clinical tool, a full-stack web architecture was implemented.

5.1 Backend Architecture (Flask)

- Framework: Python's Flask micro-framework was utilized to build the RESTful API (app.py).

- Workflow: 1. The server receives an HTTP POST request containing the 8 clinical parameters entered by the user. 2. The server script captures these inputs and formats them into a 2D NumPy array. 3. The data is preprocessed using the exact scaling methodology applied during training. 4. The preprocessed array is passed to the loaded fetalAI_model.pkl file, which executes the predict() function. 5. The resulting integer (1, 2, or 3) is translated into the corresponding string (Normal, Suspect, Pathological) and routed back to the frontend.

5.2 Frontend Interface

- UI/UX Design: The frontend was constructed using HTML5, CSS3, and JavaScript, heavily utilizing the Bootstrap framework (styles.css, scripts.js) for a responsive, mobile-friendly design.
- Diagnostic Dashboard: The index.html file serves as the primary portal. It features an intuitive form where medical staff can input the CTG values. Upon submission, the page seamlessly displays the AI-generated diagnosis, providing immediate, actionable intelligence to the healthcare provider.

6. Conclusion and Future Scope

6.1 Conclusion

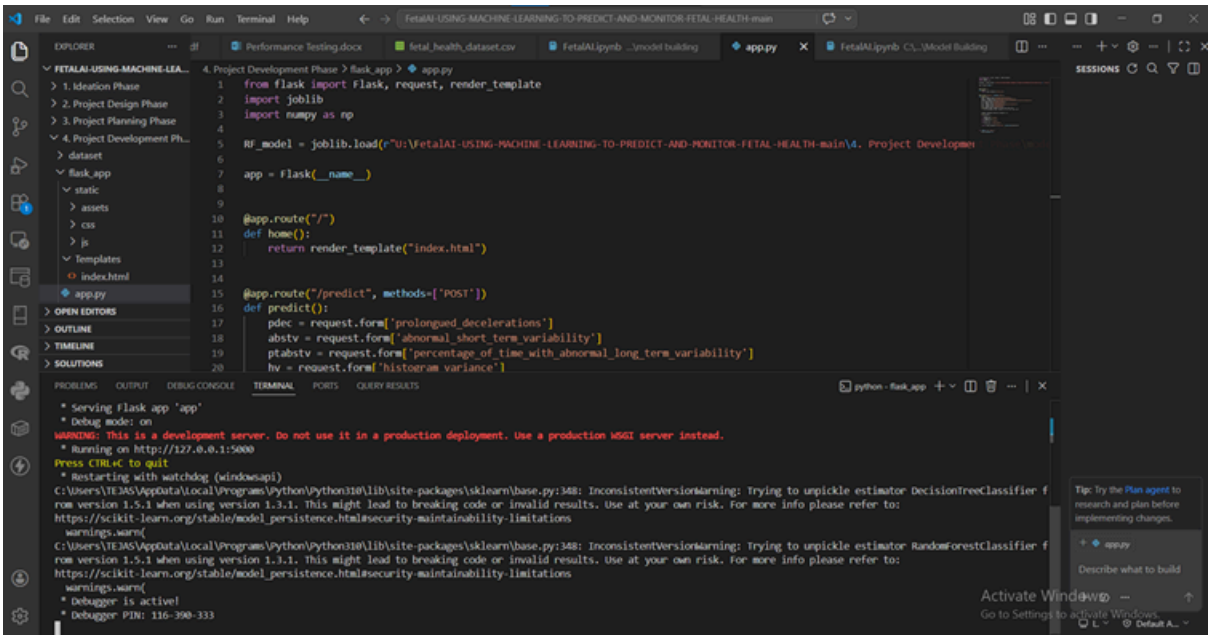
The FetalAI project successfully bridges the gap between data science and maternal-fetal medicine. By addressing severe class imbalances using SMOTE and

isolating the most predictive physiological features, the Random Forest model achieved an impressive 94.67% diagnostic accuracy. The successful deployment of this algorithmic model into a Flask-based web application demonstrates the immense potential of AI to serve as an objective, secondary diagnostic tool, ultimately assisting in the reduction of fetal mortality rates.

6.2 Future Scope

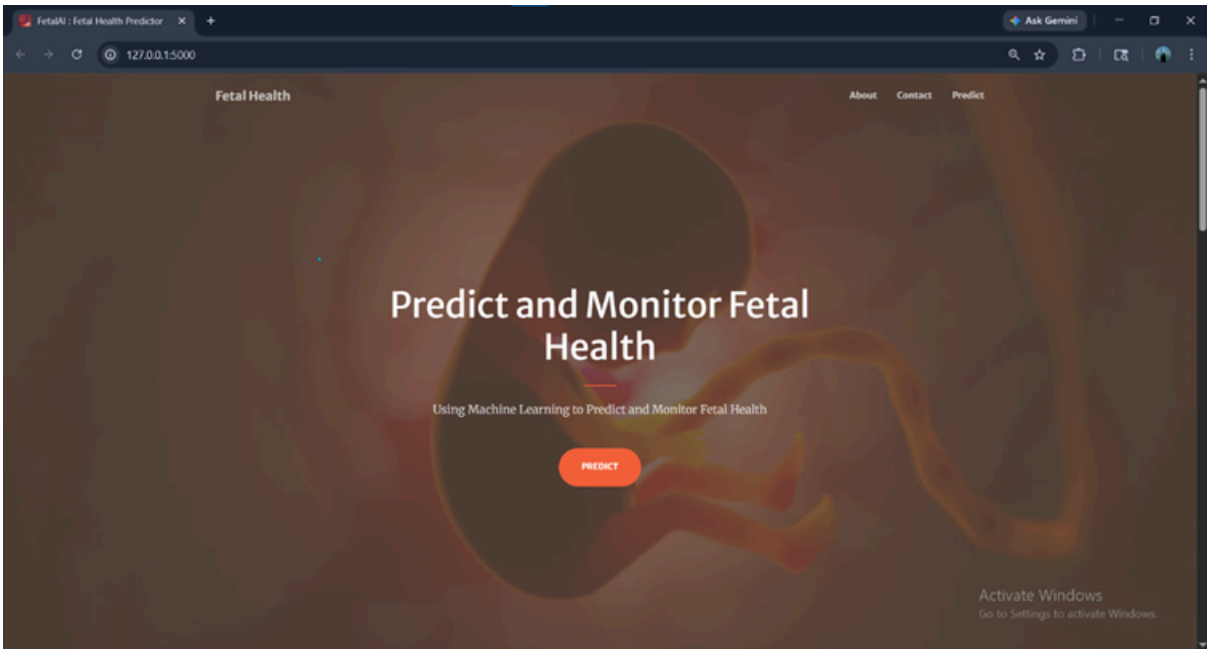
- Continuous Learning: Integration of a database (e.g., PostgreSQL or MongoDB) to store new clinical predictions, allowing the model to be periodically retrained and refined on larger, real-world datasets.
- Deep Learning Integration: Exploring Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks to analyze the continuous time-series data of the CTG directly, bypassing the need for manual feature extraction.
- Cloud Deployment: Migrating the Flask application to cloud platforms such as AWS, Heroku, or Microsoft Azure for global accessibility and scalability.

Backend:

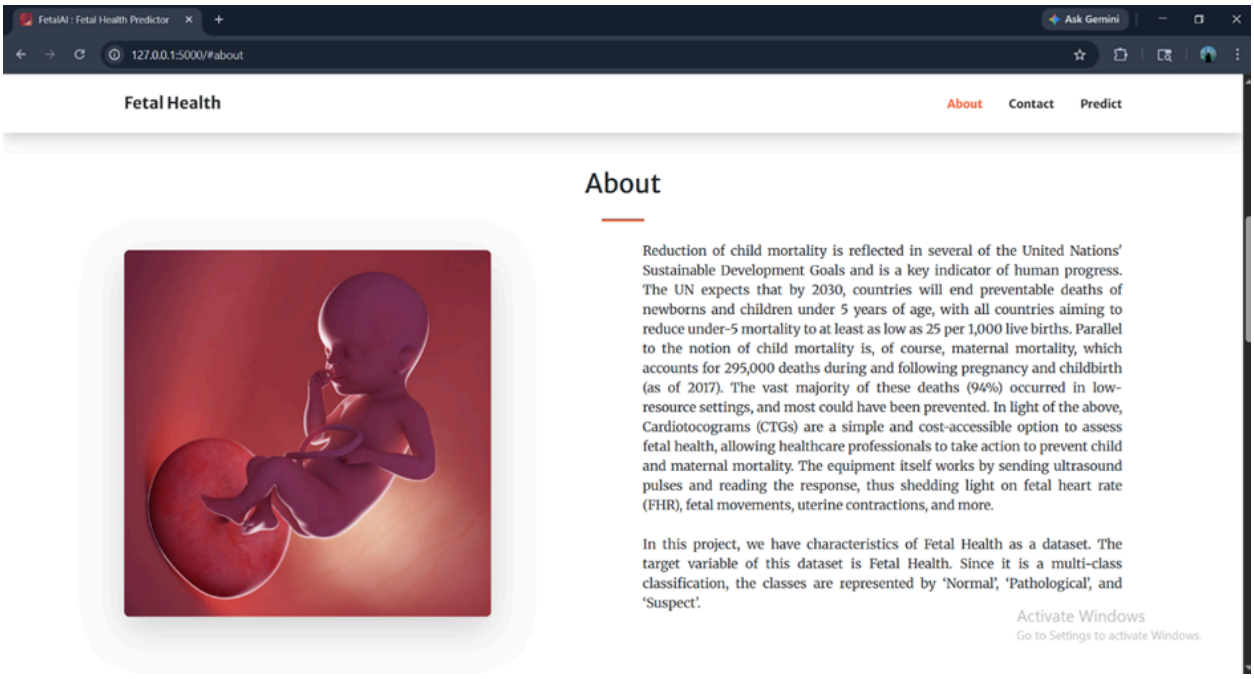


Frontend UI:

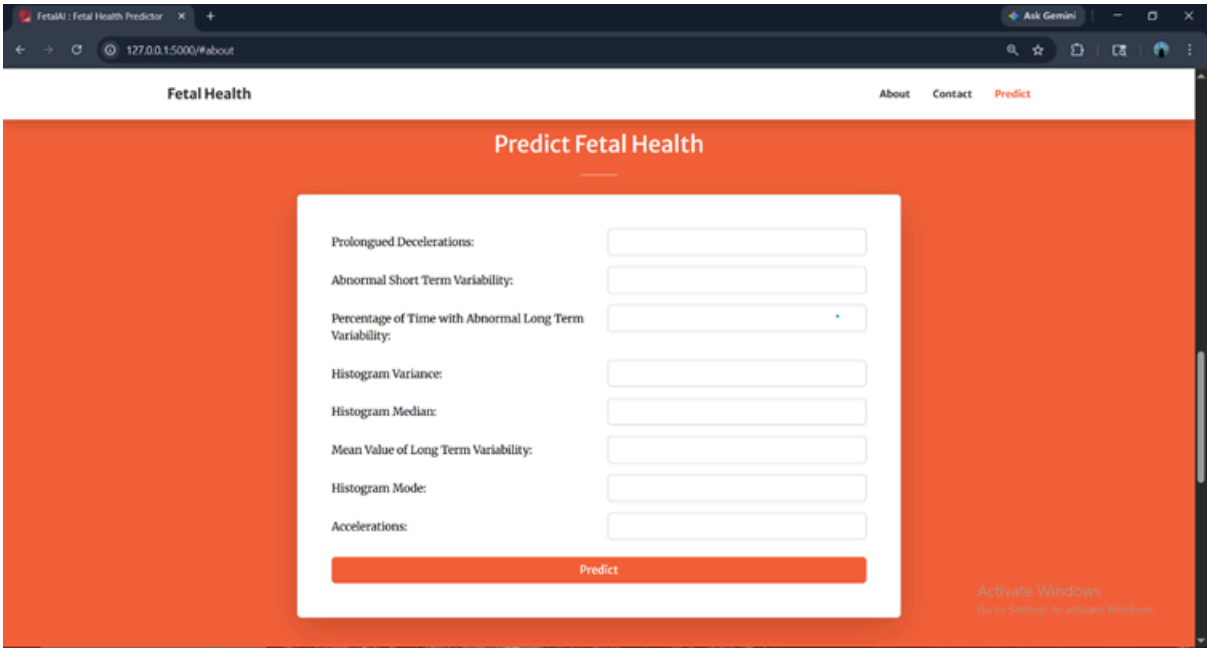
Starting Page:



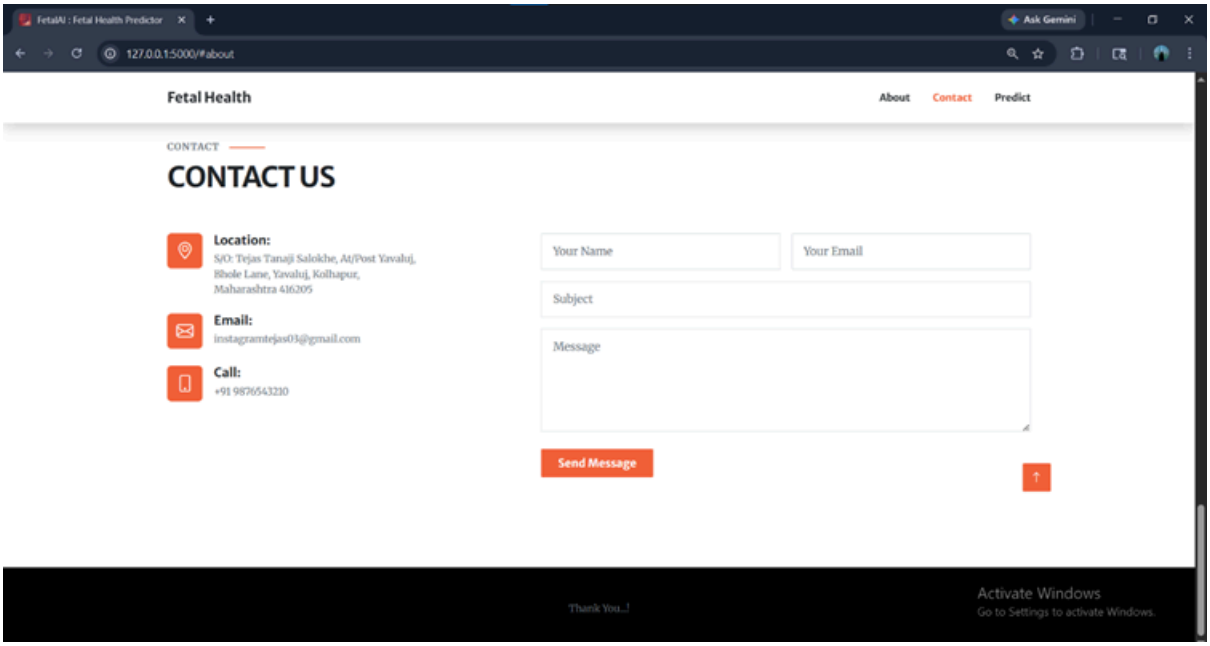
About Page:



Predict Page:

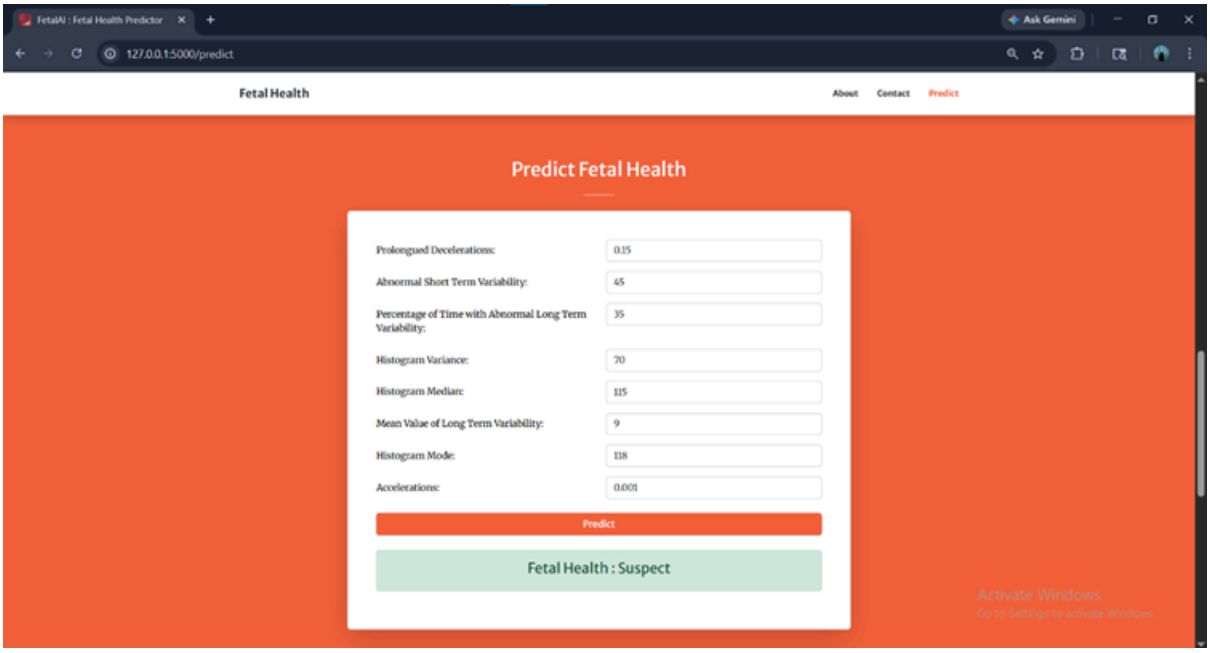


Contact Page:

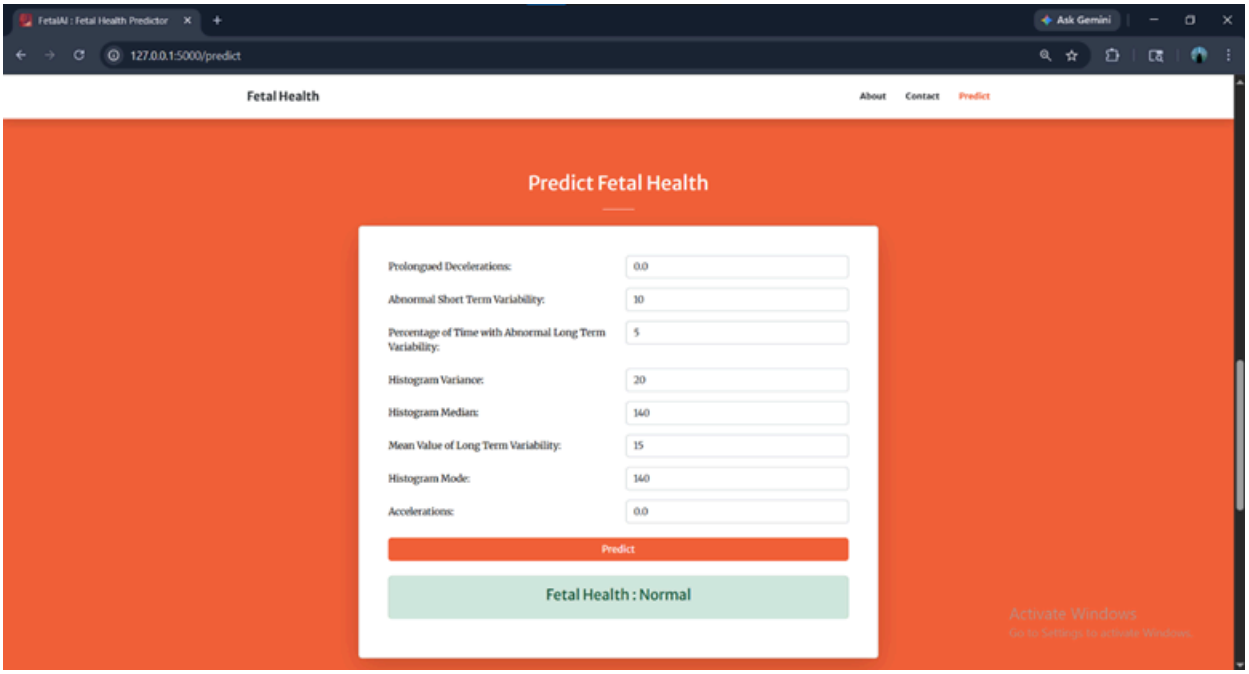
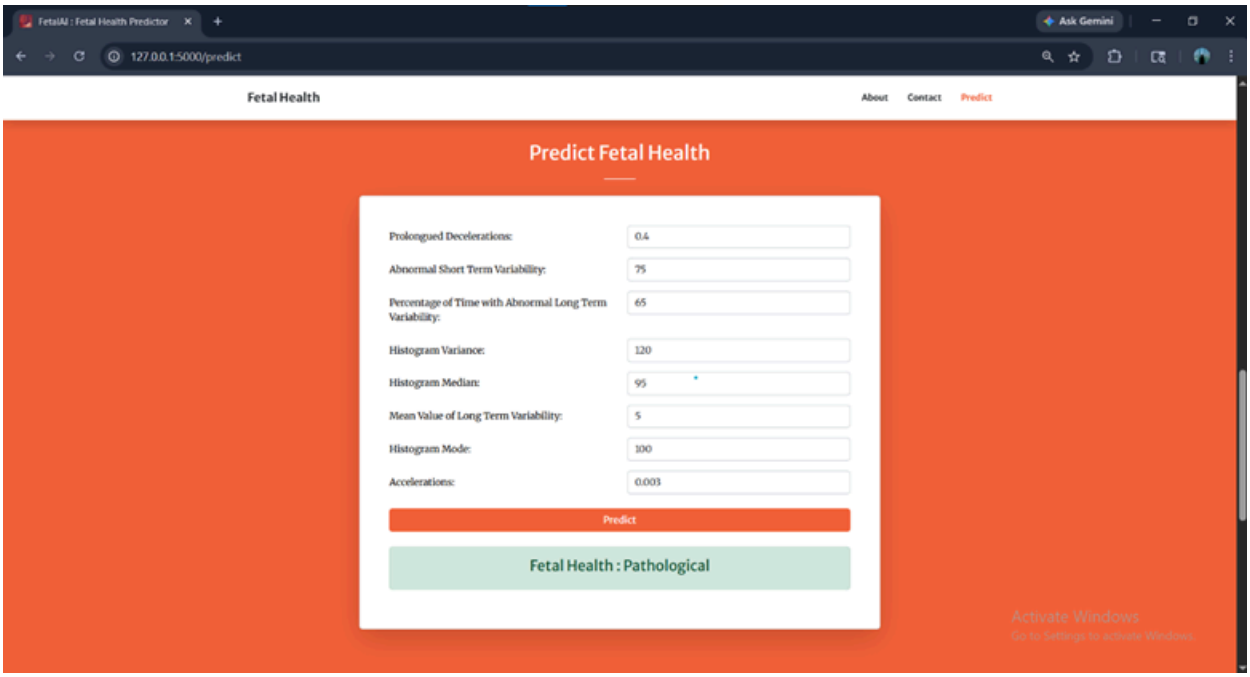


Machine Learning Pipeline:

Suspect:



Normal:



Pathological: